



École d'Ingénieurs du Littoral Côte d'Opale
(EILCO)

Master — Ingénierie des Systèmes Complexes

Mémoire de Projet de Fin d'Études (PFE)

Autoencodeurs profonds pour la réduction de
dimension
et la visualisation de données complexes

Application à la maintenance prédictive (NASA/IMS Bearing Dataset)

Auteur : Mohamed KABOURI
Lieu : Calais, France

Encadrant : M. Khalid JBILOU
Année universitaire : 2025–2026

Table des matières

1	Introduction	1
1.1	Contexte général	1
1.1.1	Notations	1
1.2	Problématique	1
1.3	Importance de la réduction de dimension	2
1.4	Pourquoi les autoencodeurs profonds ?	2
1.4.1	Principe d'un autoencodeur	3
1.5	Choix du cas d'application : maintenance prédictive	3
1.5.1	Dataset IMS Bearing (NASA)	4
1.6	Prétraitement : segmentation et MAV	4
1.6.1	Normalisation	4
1.7	Contributions du projet	4
1.8	Organisation du mémoire	5
2	État de l'art (centré autoencodeurs)	6
2.1	Réduction de dimension : PCA vs autoencodeurs	6
2.1.1	Pourquoi réduire la dimension ?	6
2.1.2	PCA : approche linéaire (formulation)	6
2.1.3	Limites de la PCA et motivation des autoencodeurs	7
2.2	Autoencodeurs profonds : AE, DAE, VAE	7
2.2.1	Autoencodeur (AE) : principe et objectif	7
2.2.2	Denoising Autoencoder (DAE)	8
2.2.3	Variational Autoencoder (VAE) (bref)	8
2.2.4	Autoencodeurs pour séries temporelles (rappel)	8
2.3	Visualisation : PCA, t-SNE et UMAP	9
2.3.1	PCA pour visualiser un espace latent	9
2.3.2	t-SNE (positionnement)	9
2.3.3	UMAP (positionnement)	9
2.4	Autoencodeurs et espace latent : interprétation	9
2.4.1	Structure de l'espace latent	9
2.4.2	Latent comme outil de visualisation (lien direct avec le sujet PFE) .	10
2.5	Application annexe : erreur de reconstruction pour la détection d'anomalies	10
2.5.1	Score d'anomalie par reconstruction	11
2.5.2	Seuils robustes : médiane et MAD	11

2.5.3	Lien avec le latent	11
3	Données et préparation	13
3.1	Données complexes : séries vibratoires multi-capteurs	13
3.1.1	Nature des données	13
3.1.2	Objectif de préparation (lien avec le sujet PFE)	13
3.2	Prétraitement : segmentation et MAV	14
3.2.1	Segmentation du signal	14
3.2.2	MAV : Mean Absolute Value (caractéristique compactante)	14
3.2.3	Justification (mathématique et pratique)	14
3.3	Mise en forme séquentielle : fenêtres (windows)	15
3.3.1	Construction des séquences d'entrée	15
3.3.2	Lien avec la réduction de dimension	15
3.4	Normalisation : scaler (standardisation)	16
3.4.1	Pourquoi normaliser ?	16
3.4.2	Forme matricielle	16
3.4.3	Cohérence train/test (point clé)	16
4	Méthodologie proposée	17
4.1	Vue d'ensemble de l'approche	17
4.2	Définition des entrées : fenêtres temporelles multi-capteurs	17
4.3	Architecture du Transformer Autoencoder	18
4.3.1	Principe général	18
4.3.2	Mécanisme d'attention (scaled dot-product)	18
4.3.3	Multi-Head Attention et bloc Transformer	18
4.4	Apprentissage : fonction de perte de reconstruction	19
4.4.1	Objectif d'optimisation	19
4.4.2	Perte MAE (Mean Absolute Error)	19
4.4.3	Rôle de la compression latente	19
4.5	Extraction du latent et visualisation par PCA	20
4.5.1	Extraction des représentations latentes	20
4.5.2	Projection PCA 2D pour la visualisation	20
4.6	Indicateur complémentaire : score MAE de reconstruction	20
4.6.1	Score par fenêtre	20
4.6.2	Seuil d'entraînement et seuil robuste (MAD)	20
4.7	Décision : NOMINAL / DÉGRADATION / PANNE	21
4.8	Synthèse et lien avec le sujet PFE	21

5	Résultats et analyse	22
5.1	Objectif des expériences et protocole	22
5.2	Qualité de représentation : comportement du latent (PCA 2D)	22
5.2.1	Projection PCA 2D et séparation des régimes	22
5.2.2	Analyse : interprétation du latent	23
5.3	Étude sur plusieurs fichiers : début vs fin (dérive temporelle)	24
5.3.1	Comparaison qualitative	24
5.3.2	Métriques synthétiques (à reporter selon tes tests)	24
5.4	Analyse du score MAE et du ratio	24
5.4.1	Évolution du score et franchissement de seuil	24
5.4.2	Rôle du ratio r	25
5.5	Cas limites et phénomènes observés	25
5.5.1	Seuil adaptatif trop élevé	25
5.5.2	Dataset shift et sensibilité aux conditions	25
5.6	Discussion : cohérence latent + reconstruction	26
6	Application et déploiement (outil de visualisation)	27
6.1	Objectifs de l'outil de visualisation	27
6.2	Architecture logicielle et pipeline de traitement	27
6.2.1	Choix technologiques	27
6.2.2	Entrées, assets et dépendances	27
6.2.3	Pipeline complet côté application	28
6.3	Interface utilisateur : organisation et paramètres	28
6.3.1	Organisation générale	28
6.3.2	Paramètres contrôlables	28
6.3.3	Sorties et interprétation	29
6.4	Robustesse : chargement du modèle et gestion des erreurs	31
6.4.1	Chargement du Transformer Autoencoder avec couche personnalisée	31
6.4.2	Validation des entrées	31
6.4.3	Extraction du latent : cas où la couche n'existe pas	31
6.5	Guide d'utilisation (pas à pas)	32
6.6	Déploiement et reproductibilité	32
6.6.1	Exécution locale	32
6.6.2	Reproductibilité	32
6.7	Limites et perspectives (côté application)	32
7	Conclusion et perspectives	34
7.1	Bilan : réduction de dimension et visualisation de données complexes	34
7.2	Apports du Transformer Autoencoder	34
7.3	Perspectives	35

7.3.1	Visualisations non linéaires : UMAP et t-SNE sur l'espace latent . .	35
7.3.2	Latents 3D et exploration interactive	35
7.3.3	Clustering et segmentation automatique dans le latent	35
7.3.4	Suivi temporel et détection de dérive (drift)	36
7.3.5	Améliorations méthodologiques	36
7.4	Conclusion générale	36

Table des figures

1	Principe de la maintenance prédictive des roulements : dégradation progressive des performances entre le point de détection précoce (Point P) et la panne fonctionnelle (Point F).	vi
2	Principe des autoencodeurs profonds pour la réduction de dimension : projection des données haute dimension vers un espace latent compact, facilitant la détection d'anomalies et la visualisation.	vii
2.1	Illustration conceptuelle : la PCA réalise une projection linéaire (peut déformer une structure non linéaire), tandis qu'un autoencodeur apprend une réduction non linéaire via un espace latent.	7
2.2	Schéma d'un autoencodeur : compression dans un espace latent de dimension réduite puis reconstruction de l'entrée.	8
2.3	Illustration comparative : PCA (linéaire) vs t-SNE/UMAP (non linéaires) pour la visualisation en 2D. Les méthodes non linéaires accentuent souvent la séparation locale (clusters), parfois au détriment des distances globales.	9
2.4	Illustration : projection PCA 2D d'un espace latent montrant une trajectoire temporelle (évolution progressive du comportement).	10
2.5	Illustration : évolution d'un score d'erreur de reconstruction (MAE) et comparaison à un seuil τ pour repérer des comportements atypiques.	12
2.6	Pipeline complet du projet : du signal vibratoire brut à l'apprentissage d'un espace latent (réduction de dimension), sa visualisation (PCA) et l'utilisation d'un score MAE (reconstruction) pour un diagnostic interprétable.	12
3.1	Évolution temporelle de la caractéristique MAV (Mean Absolute Value) pour le roulement 1. La MAV est calculée sur des segments de taille L et fournit une représentation compactée du signal vibratoire multi-capteurs.	15
4.1	Courbes d'apprentissage : évolution de la perte MAE sur l'ensemble d'entraînement (train) et de validation (val) au fil des époques.	19
5.1	Projection PCA 2D des représentations latentes z_t apprises par le Transformer Autoencoder. Les points sont colorés selon le statut (normal vs anomalie) défini à partir du score MAE et du seuil τ . Cette visualisation illustre la capacité du latent à organiser les fenêtres en régimes cohérents et à faire émerger des comportements atypiques.	23

5.2	Score d'erreur de reconstruction (MAE) en fonction des fenêtres (TIME_STEPS) et seuil τ . Les dépassements du seuil indiquent des segments dont la reconstruction s'éloigne du régime nominal appris.	25
6.1	Interface Streamlit : chargement des assets et zone d'upload.	29
6.2	Exemple : statut NOMINAL avec score MAE faible et ratio sous le seuil.	29
6.3	Capture du dashboard Streamlit en fonctionnement nominal : chargement des assets (modèle, scaler, seuil), aperçu du signal, score MAE maximal, et informations du mode debug (segments MAV, fenêtres, seuils).	30
6.4	Exemple : statut PANNE avec score MAE élevé et forte proportion de fenêtres au-dessus du seuil.	30
6.5	Projection PCA 2D des représentations latentes : séparation visuelle entre régimes normaux et atypiques.	31

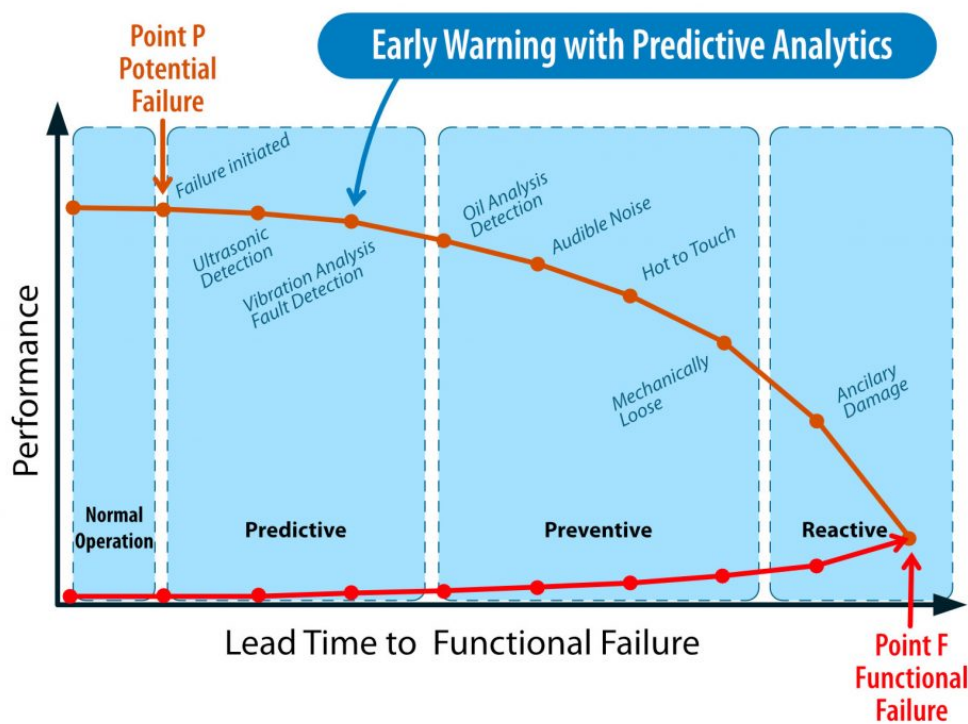


FIGURE 1 – Principe de la maintenance prédictive des roulements : dégradation progressive des performances entre le point de détection précoce (Point P) et la panne fonctionnelle (Point F).

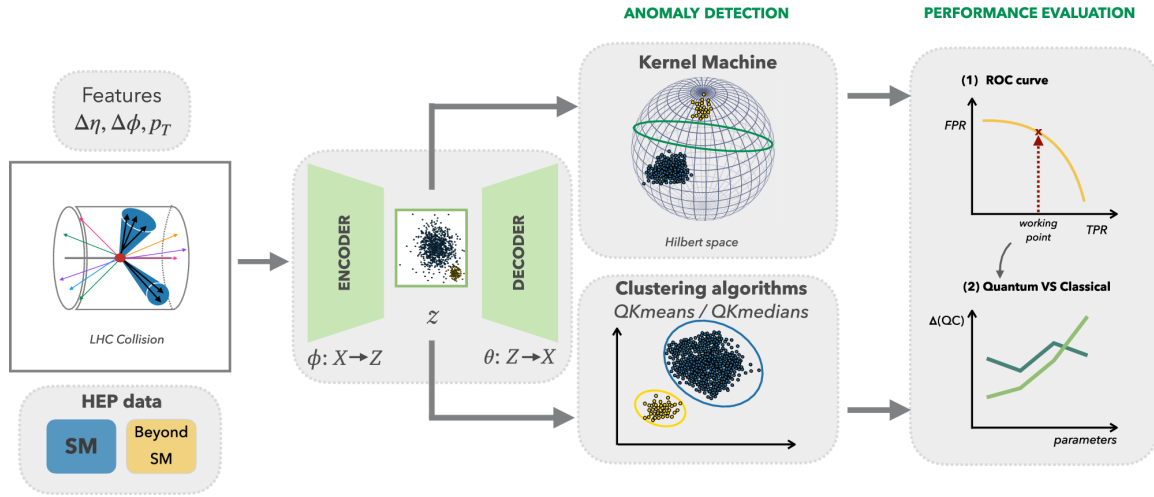


FIGURE 2 – Principe des autoencodeurs profonds pour la réduction de dimension : projection des données haute dimension vers un espace latent compact, facilitant la détection d'anomalies et la visualisation.

Chapitre 1

Introduction

1.1 Contexte général

Les systèmes modernes génèrent des volumes croissants de données issues de sources variées telles que les capteurs industriels, les systèmes IoT, les réseaux de communication ou encore les applications biomédicales. Ces données sont souvent caractérisées par :

- une **grande dimension** (nombre élevé de variables) ;
- des **relations non linéaires** complexes ;
- une **forte variabilité temporelle** ;
- la présence de **bruit** et d'incertitudes.

Dans ce contexte, l'analyse directe dans l'espace original devient difficile, voire impossible, notamment pour des tâches de compréhension globale, de visualisation ou de diagnostic. Il devient alors nécessaire de disposer de méthodes capables d'extraire des représentations plus compactes et informatives.

1.1.1 Notations

Un échantillon de données est noté :

$$x \in \mathbb{R}^d,$$

où d représente la dimension du vecteur d'observation. Un jeu de données est défini par :

$$\mathcal{D} = \{x^{(i)}\}_{i=1}^N.$$

Dans le cas des séries temporelles multi-capteurs, une observation correspond à une séquence :

$$X \in \mathbb{R}^{T \times d},$$

où T est la longueur de la fenêtre temporelle et d le nombre de capteurs.

1.2 Problématique

Le sujet de ce Projet de Fin d'Études est :

Autoencodeurs profonds pour la réduction de dimension et la visualisation de données complexes.

La problématique centrale peut être formulée comme suit :

Comment apprendre automatiquement une représentation compacte et pertinente de données complexes, tout en conservant leur structure essentielle et en permettant une visualisation exploitable ?

Mathématiquement, la réduction de dimension consiste à apprendre une application :

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^k, \quad k \ll d,$$

où $z = \phi(x)$ est une représentation latente de faible dimension. L'objectif est que cette représentation conserve l'information essentielle contenue dans x , notamment en termes de structure, de voisinage et de dynamique.

1.3 Importance de la réduction de dimension

La réduction de dimension joue un rôle fondamental dans l'analyse de données complexes. Lorsque la dimension d est élevée, plusieurs phénomènes apparaissent, regroupés sous le terme de *malédiction de la dimension* :

- les distances entre points deviennent peu discriminantes ;
- les visualisations directes sont impossibles au-delà de trois dimensions ;
- les algorithmes d'apprentissage nécessitent davantage de données pour généraliser ;
- le bruit peut masquer les structures pertinentes.

Réduire la dimension permet donc :

- de faciliter l'exploration et la visualisation des données ;
- d'extraire les facteurs latents gouvernant le système ;
- d'améliorer la robustesse et l'interprétabilité des modèles.

Dans ce projet, la réduction de dimension n'est pas uniquement vue comme une compression, mais comme un **outil d'analyse et de compréhension** du comportement des données.

1.4 Pourquoi les autoencodeurs profonds ?

Les méthodes classiques telles que l'Analyse en Composantes Principales (PCA) reposent sur des projections linéaires. Bien qu'efficaces et interprétables, elles présentent des limites lorsque les données reposent sur des structures non linéaires.

Or, dans de nombreux systèmes réels, les données vivent sur des variétés non linéaires (*manifolds*). Une projection linéaire peut alors déformer la structure des données et masquer

certaines dynamiques importantes.

Les autoencodeurs profonds permettent d'apprendre une réduction de dimension **non linéaire** directement à partir des données, sans hypothèse forte sur leur structure. Ils offrent ainsi un cadre flexible pour :

- apprendre un espace latent expressif ;
- préserver les relations complexes entre variables ;
- exploiter ce latent pour la visualisation et l'analyse.

1.4.1 Principe d'un autoencodeur

Un autoencodeur est composé de deux fonctions paramétrées :

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^k \quad (\text{encodeur}),$$

$$g_{\phi} : \mathbb{R}^k \rightarrow \mathbb{R}^d \quad (\text{décodeur}).$$

Pour une entrée x , on définit :

$$z = f_{\theta}(x), \quad \hat{x} = g_{\phi}(z).$$

Les paramètres (θ, ϕ) sont appris en minimisant une perte de reconstruction :

$$\min_{\theta, \phi} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(x^{(i)}, \hat{x}^{(i)}).$$

Dans ce projet, une perte de type MAE (Mean Absolute Error) est privilégiée pour sa robustesse au bruit.

1.5 Choix du cas d'application : maintenance prédictive

La maintenance prédictive vise à anticiper les défaillances des systèmes industriels avant qu'elles ne surviennent, en exploitant des données issues de capteurs. Contrairement à la maintenance corrective ou préventive, elle repose sur l'analyse continue du comportement réel du système.

Les machines tournantes (roulements, moteurs) sont particulièrement adaptées à ce type d'approche, car les défauts mécaniques se manifestent souvent par :

- une augmentation progressive de l'énergie vibratoire ;
- des dérives lentes difficiles à détecter ;
- des changements subtils de régime.

Ces signaux sont complexes, non stationnaires et de grande dimension, ce qui en fait

un cas d'étude idéal pour les méthodes de réduction de dimension et de visualisation.

1.5.1 Dataset IMS Bearing (NASA)

Le dataset IMS Bearing, développé par la NASA et l'Université de l'Iowa, est une référence académique en maintenance prédictive. Il présente plusieurs avantages :

- données réelles issues de bancs d'essai industriels ;
- évolution progressive jusqu'à la défaillance ;
- absence d'annotations fines, justifiant une approche non supervisée ;
- large utilisation dans la littérature scientifique.

Il permet ainsi d'évaluer la capacité d'un espace latent à capturer les dérives et transitions du comportement du système.

1.6 Prétraitement : segmentation et MAV

Les signaux vibratoires bruts sont volumineux et difficilement exploitables directement. Une étape de prétraitement est donc appliquée.

Soit un signal discret $s[n]$, découpé en segments de taille L . Pour le segment m , on définit le **Mean Absolute Value (MAV)** :

$$\text{MAV}(m) = \frac{1}{L} \sum_{n=mL}^{(m+1)L-1} |s[n]|.$$

Dans le cas multi-capteurs (d capteurs), on obtient :

$$x_m = (\text{MAV}_1(m), \dots, \text{MAV}_d(m)) \in \mathbb{R}^d.$$

Ces vecteurs sont ensuite organisés en fenêtres temporelles :

$$X_t = [x_t, x_{t+1}, \dots, x_{t+T-1}] \in \mathbb{R}^{T \times d}.$$

1.6.1 Normalisation

Afin de stabiliser l'apprentissage, une normalisation est appliquée :

$$\tilde{x} = \frac{x - \mu}{\sigma},$$

où μ et σ sont estimés sur les données d'entraînement nominales.

1.7 Contributions du projet

Les principales contributions de ce travail sont :

- l'apprentissage d'un **espace latent non linéaire** via un Transformer Autoencoder ;
- l'exploitation du latent pour la **réduction de dimension et la visualisation** ;
- l'utilisation de l'erreur de reconstruction (MAE) comme indicateur complémentaire ;
- le développement d'un **outil interactif** pour l'exploration et le diagnostic.

1.8 Organisation du mémoire

Le mémoire est structuré comme suit :

- Chapitre 2 : état de l'art ;
- Chapitre 3 : données et préparation ;
- Chapitre 4 : méthodologie proposée ;
- Chapitre 5 : résultats et analyse ;
- Chapitre 6 : application et déploiement ;
- Chapitre 7 : conclusion et perspectives.

Chapitre 2

État de l'art (centré autoencodeurs)

2.1 Réduction de dimension : PCA vs autoencodeurs

2.1.1 Pourquoi réduire la dimension ?

Soit un jeu de données $\mathcal{D} = \{x^{(i)}\}_{i=1}^N$ avec $x^{(i)} \in \mathbb{R}^d$, où d peut être grand. L'objectif de la réduction de dimension est de construire une représentation

$$z = \phi(x) \in \mathbb{R}^k, \quad k \ll d,$$

qui conserve autant que possible l'information utile (structure, voisinages, variabilité), tout en facilitant l'analyse et la visualisation (souvent en 2D/3D).

2.1.2 PCA : approche linéaire (formulation)

La PCA (Analyse en Composantes Principales) cherche une projection linéaire vers un sous-espace de dimension k maximisant la variance projetée.

On centre d'abord les données :

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x^{(i)}, \quad \tilde{x}^{(i)} = x^{(i)} - \bar{x}.$$

La matrice de covariance est :

$$\Sigma = \frac{1}{N} \sum_{i=1}^N \tilde{x}^{(i)} \tilde{x}^{(i)\top} \in \mathbb{R}^{d \times d}.$$

On calcule les vecteurs propres $(u_1, \dots, u_k)$ associés aux plus grandes valeurs propres de Σ , et on définit la projection PCA :

$$z = U_k^\top \tilde{x} \in \mathbb{R}^k, \quad \text{où } U_k = [u_1, \dots, u_k] \in \mathbb{R}^{d \times k}.$$

La variance expliquée par la composante j est donnée par

$$\text{EVR}_j = \frac{\lambda_j}{\sum_{\ell=1}^d \lambda_\ell},$$

où λ_j est la j -ème valeur propre (triée décroissante).

2.1.3 Limites de la PCA et motivation des autoencodeurs

La PCA est efficace et interprétable mais **linéaire**. Si les données vivent sur une **variété non linéaire** (*manifold*), une projection linéaire peut déformer la structure (voisinages, séparations, trajectoires). Les autoencodeurs profonds permettent d'apprendre une réduction de dimension **non linéaire** en optimisant une reconstruction.

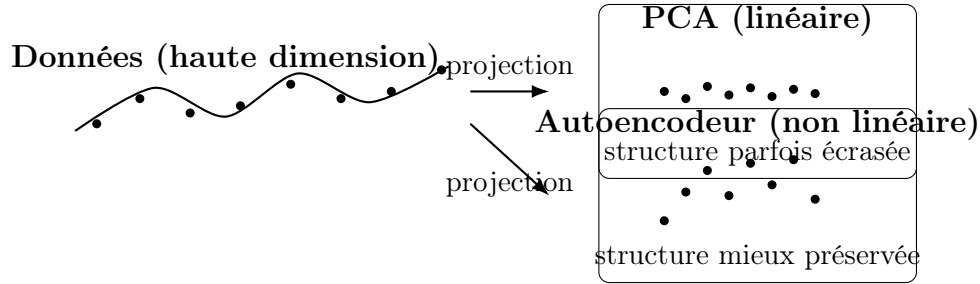


FIGURE 2.1 – Illustration conceptuelle : la PCA réalise une projection linéaire (peut déformer une structure non linéaire), tandis qu'un autoencodeur apprend une réduction non linéaire via un espace latent.

2.2 Autoencodeurs profonds : AE, DAE, VAE

2.2.1 Autoencodeur (AE) : principe et objectif

Un autoencodeur apprend deux fonctions paramétrées :

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^k \quad (\text{encodeur}), \quad g_{\phi} : \mathbb{R}^k \rightarrow \mathbb{R}^d \quad (\text{décodeur}),$$

telles que, pour une entrée x :

$$z = f_{\theta}(x), \quad \hat{x} = g_{\phi}(z) = g_{\phi}(f_{\theta}(x)).$$

L'apprentissage minimise une perte de reconstruction :

$$\min_{\theta, \phi} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(x^{(i)}, \hat{x}^{(i)}).$$

Exemples de pertes usuelles :

$$\mathcal{L}_{\text{MSE}}(x, \hat{x}) = \|x - \hat{x}\|_2^2, \quad \mathcal{L}_{\text{MAE}}(x, \hat{x}) = \|x - \hat{x}\|_1.$$

Le cas *undercomplete* ($k \ll d$) force le modèle à compresser l'information dans l'espace latent.

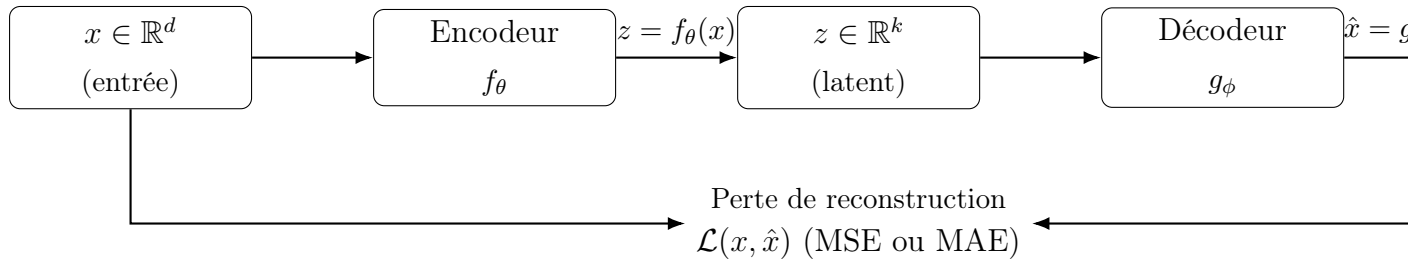


FIGURE 2.2 – Schéma d'un autoencodeur : compression dans un espace latent de dimension réduite puis reconstruction de l'entrée.

2.2.2 Denoising Autoencoder (DAE)

Le DAE apprend à reconstruire une entrée propre à partir d'une version bruitée. On définit une corruption $\tilde{x} \sim q(\tilde{x} | x)$ (bruit gaussien, masquage, etc.), et on optimise :

$$\min_{\theta, \phi} \mathbb{E}_{\tilde{x} \sim q(\tilde{x} | x)} [\mathcal{L}(x, g_{\phi}(f_{\theta}(\tilde{x})))].$$

Cette formulation agit comme une régularisation, rendant la représentation latente plus robuste au bruit.

2.2.3 Variational Autoencoder (VAE) (bref)

Le VAE est un modèle génératif probabiliste. L'encodeur approxime une distribution

$$q_{\theta}(z | x) = \mathcal{N}(\mu_{\theta}(x), \text{diag}(\sigma_{\theta}(x)^2)),$$

et le décodeur définit une vraisemblance $p_{\phi}(x | z)$. L'apprentissage maximise l'ELBO :

$$\mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_{\theta}(z | x)} [\log p_{\phi}(x | z)] - \text{KL}(q_{\theta}(z | x) \parallel p(z)),$$

avec $p(z) = \mathcal{N}(0, I)$ en général. Le terme KL structure l'espace latent, facilitant parfois la visualisation/interpolation.

2.2.4 Autoencodeurs pour séries temporelles (rappel)

Pour des séquences $X \in \mathbb{R}^{T \times d}$, l'encodeur et le décodeur opèrent sur des dépendances temporelles. Plusieurs familles existent : LSTM-AE, TCN-AE et **Transformer-AE**. Le Transformer repose sur l'attention, permettant de capturer des corrélations globales dans la fenêtre :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Cette capacité motive son utilisation dans ce projet.

2.3 Visualisation : PCA, t-SNE et UMAP

2.3.1 PCA pour visualiser un espace latent

Une fois un latent $z \in \mathbb{R}^k$ appris, on peut visualiser en 2D par PCA :

$$y = Pz, \quad y \in \mathbb{R}^2,$$

où $P \in \mathbb{R}^{2 \times k}$ contient les deux directions principales estimées sur les latents.

2.3.2 t-SNE (positionnement)

t-SNE cherche une représentation basse dimension qui préserve surtout les voisinages. Il définit des probabilités de voisinage p_{ij} en haute dimension et q_{ij} en basse dimension, puis minimise une divergence KL :

$$\min \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right).$$

t-SNE produit souvent des clusters visuellement séparés, mais peut déformer les distances globales (et est sensible aux hyperparamètres).

2.3.3 UMAP (positionnement)

UMAP modélise la structure de voisinage via un graphe (fuzzy simplicial set) et optimise une fonction qui préserve la topologie locale. Dans la pratique, UMAP est souvent plus rapide que t-SNE et conserve mieux certaines structures globales. Il est utile pour visualiser des latents, notamment quand le nombre d'exemples est grand.

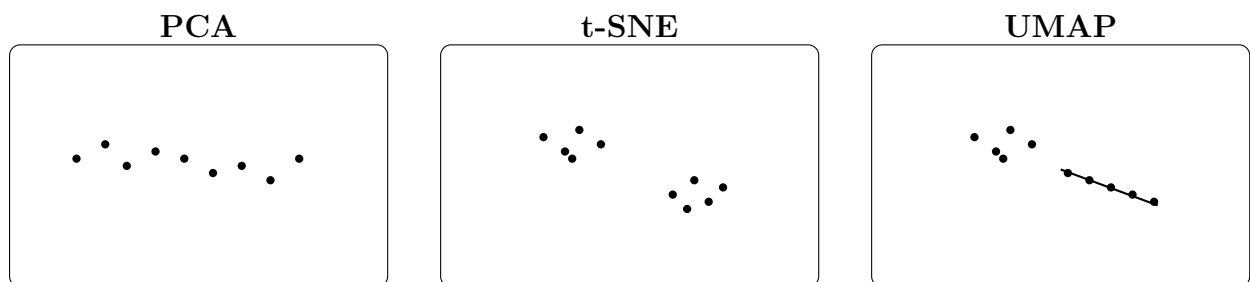


FIGURE 2.3 – Illustration comparative : PCA (linéaire) vs t-SNE/UMAP (non linéaires) pour la visualisation en 2D. Les méthodes non linéaires accentuent souvent la séparation locale (clusters), parfois au détriment des distances globales.

2.4 Autoencodeurs et espace latent : interprétation

2.4.1 Structure de l'espace latent

L'espace latent $z = f_{\theta}(x)$ doit idéalement organiser les données de façon cohérente :

- des observations similaires en entrée doivent être proches en latent ;
- les variations principales doivent correspondre à des directions (ou régions) interprétables ;
- dans le cas temporel, on peut observer une *trajectoire* dans le latent au fil des fenêtres.

Une mesure simple de proximité en latent est la distance euclidienne :

$$d(z_i, z_j) = \|z_i - z_j\|_2.$$

On peut aussi analyser la variance expliquée après PCA, ou appliquer un clustering (k-means) sur z pour observer des regroupements.

2.4.2 Latent comme outil de visualisation (lien direct avec le sujet PFE)

Dans ce PFE, la contribution principale est l'usage du latent pour **réduire la dimension** et **visualiser** l'évolution des données. La projection PCA 2D des latents permet d'observer :

- des clusters (régimes de fonctionnement),
- des transitions (début de dérive),
- des points isolés (comportements atypiques).

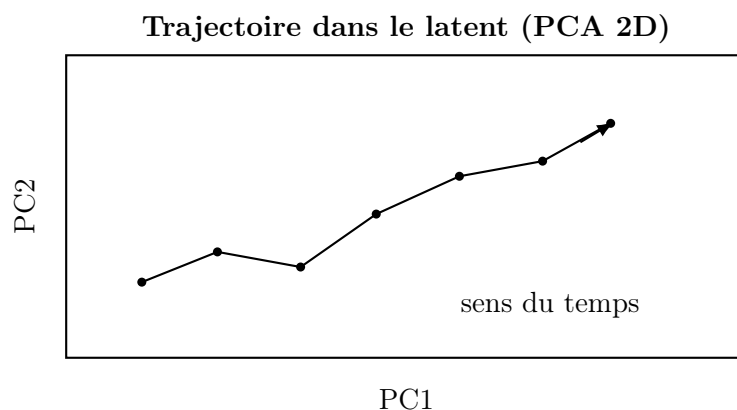


FIGURE 2.4 – Illustration : projection PCA 2D d'un espace latent montrant une trajectoire temporelle (évolution progressive du comportement).

2.5 Application annexe : erreur de reconstruction pour la détection d'anomalies

2.5.1 Score d'anomalie par reconstruction

Même si le cœur du sujet est la réduction de dimension et la visualisation, un usage naturel des autoencodeurs est l'analyse de la reconstruction. Pour une séquence $X \in \mathbb{R}^{T \times d}$, on note $\hat{X}$ sa reconstruction. Le score MAE utilisé dans ce projet est :

$$s(X) = \text{MAE}(X, \hat{X}) = \frac{1}{Td} \sum_{t=1}^T \sum_{j=1}^d |X_{t,j} - \hat{X}_{t,j}|.$$

Un score élevé suggère que X s'éloigne de la distribution apprise (souvent nominale).

2.5.2 Seuils robustes : médiane et MAD

Pour définir un seuil robuste, on peut utiliser la médiane et la *Median Absolute Deviation* (MAD) :

$$\text{MAD}(s) = \text{median}\left(|s - \text{median}(s)|\right).$$

Un seuil adaptatif peut être défini par :

$$\tau_{\text{MAD}} = \text{median}(s) + k \cdot \text{MAD}(s),$$

où $k > 0$ contrôle la sensibilité. Dans le cadre du projet, ce seuil peut être calculé sur une *baseline* (début du fichier) pour éviter que le seuil ne suive une panne déjà installée.

2.5.3 Lien avec le latent

Un point important est la cohérence entre :

- la visualisation du latent (points isolés, changements de structure),
- et le score de reconstruction (hausse de MAE).

L'intérêt de combiner les deux est d'obtenir un diagnostic plus interprétable : le latent explique *où* se situent les données, et la MAE indique à *quel point* elles s'écartent du comportement appris.

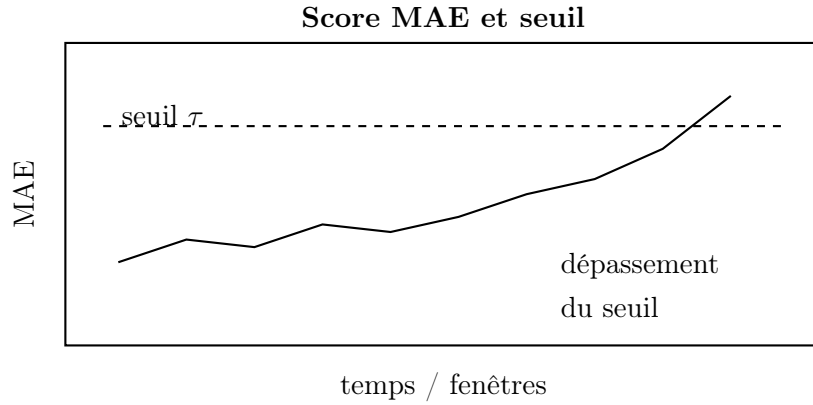


FIGURE 2.5 – Illustration : évolution d'un score d'erreur de reconstruction (MAE) et comparaison à un seuil τ pour repérer des comportements atypiques.

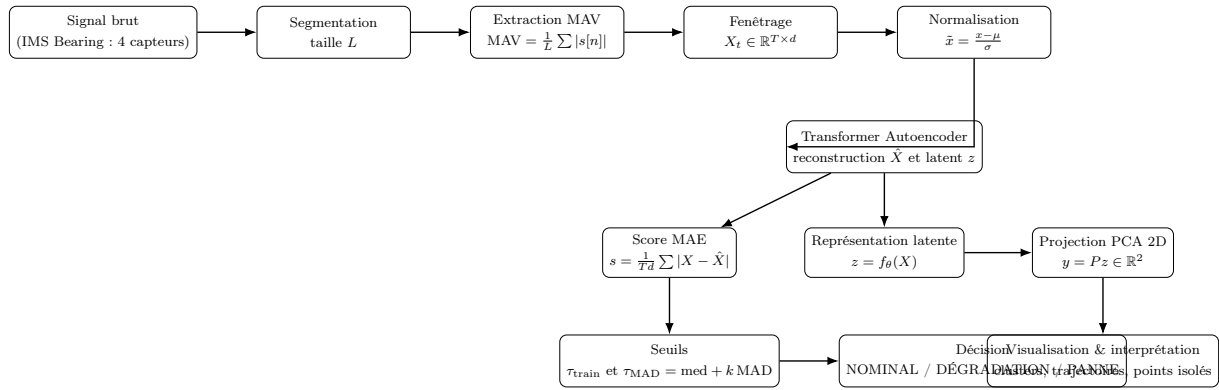


FIGURE 2.6 – Pipeline complet du projet : du signal vibratoire brut à l'apprentissage d'un espace latent (réduction de dimension), sa visualisation (PCA) et l'utilisation d'un score MAE (reconstruction) pour un diagnostic interprétable.

Chapitre 3

Données et préparation

3.1 Données complexes : séries vibratoires multi-capteurs

3.1.1 Nature des données

Les données étudiées proviennent d'un système de surveillance vibratoire de roulements, où plusieurs capteurs mesurent des signaux temporels. À chaque instant discret n , on observe un vecteur multi-capteurs :

$$x[n] = (x_1[n], x_2[n], \dots, x_d[n]) \in \mathbb{R}^d,$$

où d représente le nombre de canaux (dans notre cas $d = 4$). Pour un fichier donné, on dispose d'une séquence de longueur N :

$$X = \{x[n]\}_{n=1}^N.$$

Ces données sont dites *complexes* car elles combinent :

- une forte dimension temporelle (grand N) ;
- plusieurs capteurs (dimension d) ;
- des structures non stationnaires possibles (évolution progressive du comportement mécanique).

3.1.2 Objectif de préparation (lien avec le sujet PFE)

L'objectif n'est pas seulement de détecter des anomalies, mais surtout de construire une représentation compacte et visualisable. La préparation des données vise donc à transformer le signal brut en une séquence de vecteurs de caractéristiques plus stables, puis en fenêtres adaptées à l'apprentissage d'un autoencodeur temporel. On cherche une représentation $z \in \mathbb{R}^k$ (avec $k \ll dT$) telle que :

$$z = f_\theta(X_t), \quad X_t \in \mathbb{R}^{T \times d},$$

afin de permettre la visualisation (PCA sur z) et l'analyse de la reconstruction.

3.2 Prétraitement : segmentation et MAV

3.2.1 Segmentation du signal

Le signal brut est très long et bruité. Une étape classique consiste à le découper en segments contigus de taille L échantillons. Pour le segment m (indexé à partir de 0), on définit :

$$x_j^{(m)}[n] = x_j[mL + n], \quad n = 0, \dots, L-1, \quad j = 1, \dots, d.$$

Chaque segment correspond ainsi à une fenêtre courte de signal brut sur laquelle on calcule des caractéristiques.

3.2.2 MAV : Mean Absolute Value (caractéristique compactante)

Pour compacter l'information et réduire fortement la dimension, on utilise la *Mean Absolute Value* (MAV). Pour un capteur j et un segment m :

$$\text{MAV}_j(m) = \frac{1}{L} \sum_{n=0}^{L-1} |x_j^{(m)}[n]|.$$

On obtient alors, pour chaque segment, un vecteur de caractéristiques :

$$v(m) = (\text{MAV}_1(m), \dots, \text{MAV}_d(m)) \in \mathbb{R}^d.$$

Si le signal contient N échantillons, le nombre de segments est

$$M = \left\lfloor \frac{N}{L} \right\rfloor,$$

et la série de caractéristiques est $V = \{v(m)\}_{m=0}^{M-1}$.

3.2.3 Justification (mathématique et pratique)

La MAV réalise deux effets importants :

- **réduction de dimension** : on passe de $L \times d$ valeurs brutes par segment à seulement d valeurs ;
- **stabilisation** : l'opérateur moyenne réduit l'influence du bruit rapide et conserve une mesure d'amplitude.

Du point de vue mathématique, la MAV est un estimateur de l'amplitude moyenne au sens L^1 sur le segment, ce qui fournit une représentation plus compacte et adaptée à l'apprentissage d'un espace latent.

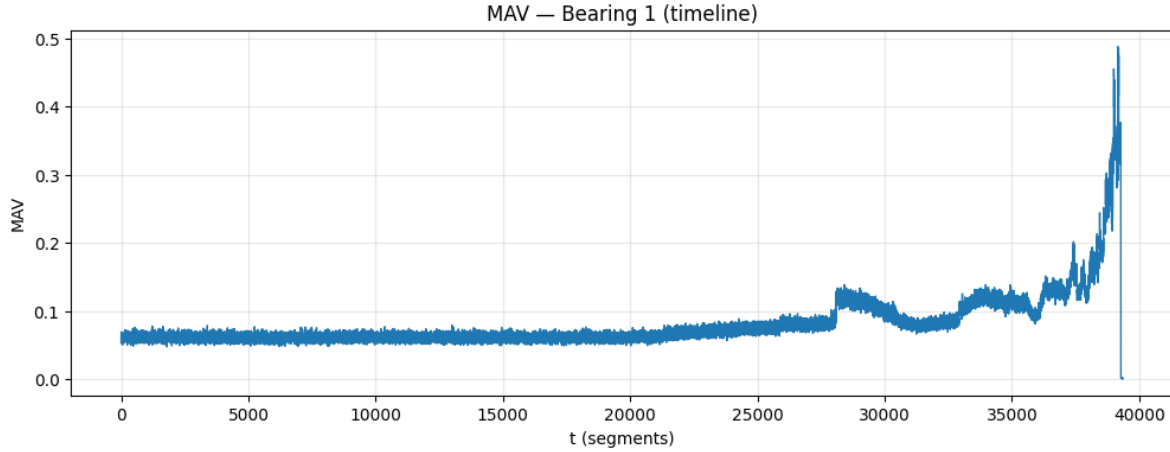


FIGURE 3.1 – Évolution temporelle de la caractéristique MAV (Mean Absolute Value) pour le roulement 1. La MAV est calculée sur des segments de taille L et fournit une représentation compactée du signal vibratoire multi-capteurs.

3.3 Mise en forme séquentielle : fenêtres (windows)

3.3.1 Construction des séquences d'entrée

L'autoencodeur temporel (Transformer-AE) attend des entrées de dimension fixe $T \times d$, où T est la longueur de la fenêtre (nombre de segments MAV consécutifs). À partir de la série $V = \{v(m)\}_{m=0}^{M-1}$, on construit des fenêtres glissantes :

$$X_t = [v(t), v(t+1), \dots, v(t+T-1)] \in \mathbb{R}^{T \times d}, \quad t = 0, \dots, M-T.$$

Le jeu de données final est donc :

$$\mathcal{X} = \{X_t\}_{t=0}^{M-T}.$$

Chaque X_t représente l'évolution temporelle locale de l'amplitude moyenne (MAV) sur T segments.

3.3.2 Lien avec la réduction de dimension

Chaque fenêtre X_t appartient à $\mathbb{R}^{T \times d}$ (dimension Td après vectorisation). L'encodeur apprend alors une projection non linéaire :

$$z_t = f_\theta(X_t) \in \mathbb{R}^k, \quad k \ll Td,$$

ce qui répond directement au sujet du PFE : *apprendre un espace latent compact et visualisable*.

3.4 Normalisation : scaler (standardisation)

3.4.1 Pourquoi normaliser ?

Les capteurs peuvent avoir des amplitudes et des variances différentes. Sans normalisation, l'autoencodeur risque de donner plus d'importance aux canaux de plus forte variance. On applique donc une standardisation (apprise sur l'ensemble d'entraînement) :

$$\tilde{v}_j(m) = \frac{v_j(m) - \mu_j}{\sigma_j},$$

où μ_j et σ_j sont la moyenne et l'écart-type estimés sur le jeu d'apprentissage.

3.4.2 Forme matricielle

En notant $v(m) \in \mathbb{R}^d$ et $\mu, \sigma \in \mathbb{R}^d$, la transformation s'écrit composante par composante :

$$\tilde{v}(m) = (v(m) - \mu) \oslash \sigma,$$

où $\oslash$ désigne la division terme à terme. Les fenêtres d'entrée utilisées par le modèle sont alors construites à partir des $\tilde{v}(m)$.

3.4.3 Cohérence train/test (point clé)

Le même scaler (`scaler_bearing.gz`) est appliqué :

- pendant l'entraînement (pour estimer μ, σ) ;
- lors des tests et du déploiement (dashboard Streamlit).

Cela garantit la cohérence de la distribution d'entrée du modèle entre entraînement et utilisation.

Chapitre 4

Méthodologie proposée

4.1 Vue d'ensemble de l'approche

L'objectif central de ce PFE est la **réduction de dimension** et la **visualisation** de données complexes au moyen d'autoencodeurs profonds. À partir des fenêtres normalisées $X_t \in \mathbb{R}^{T \times d}$ construites au Chapitre 3, nous entraînons un **Transformer Autoencoder** afin d'apprendre un espace latent de faible dimension :

$$z_t = f_\theta(X_t) \in \mathbb{R}^k, \quad k \ll Td,$$

et une reconstruction associée :

$$\hat{X}_t = g_\phi(z_t) = g_\phi(f_\theta(X_t)).$$

L'espace latent z_t sert de représentation compacte (réduction de dimension) et supporte la visualisation (projection PCA). En complément, l'erreur de reconstruction (MAE) fournit un indicateur utile pour repérer des écarts au comportement appris.

4.2 Définition des entrées : fenêtres temporelles multi-capteurs

Chaque observation d'entrée correspond à une **fenêtre** de longueur T construite à partir des caractéristiques MAV (Chapitre 3), avec d capteurs :

$$X_t = \begin{bmatrix} v(t) \\ v(t+1) \\ \vdots \\ v(t+T-1) \end{bmatrix} \in \mathbb{R}^{T \times d},$$

où $v(t) \in \mathbb{R}^d$ est le vecteur MAV au segment t . Après normalisation par le scaler appris sur l'entraînement, on obtient la séquence normalisée $\tilde{X}_t$, qui est utilisée comme entrée du modèle.

4.3 Architecture du Transformer Autoencoder

4.3.1 Principe général

Un autoencodeur est constitué de deux réseaux :

- un **encodeur** f_θ qui projette l'entrée vers une représentation compacte z ;
- un **décodeur** g_ϕ qui reconstruit l'entrée $\hat{X}$ à partir de z .

Formellement :

$$z = f_\theta(X), \quad \hat{X} = g_\phi(z).$$

Dans notre cas, l'encodeur et le décodeur exploitent des blocs Transformer afin de modéliser des dépendances temporelles au sein de la fenêtre.

4.3.2 Mécanisme d'attention (scaled dot-product)

Soit $X \in \mathbb{R}^{T \times d}$ une fenêtre en entrée. On projette X en *queries*, *keys* et *values* :

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

où W_Q, W_K, W_V sont des matrices apprises. L'attention est alors définie par :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

avec d_k la dimension des clés. Cette opération permet à chaque pas temporel d'agréger l'information des autres pas, et donc de capturer des corrélations globales dans une fenêtre.

4.3.3 Multi-Head Attention et bloc Transformer

Le Transformer utilise plusieurs têtes d'attention (*multi-head*) pour capturer différentes relations :

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O.$$

Un bloc Transformer standard combine :

- Multi-Head Attention,
- une couche feed-forward positionnelle,
- des connexions résiduelles et des normalisations (LayerNorm),
- du dropout pour limiter le sur-apprentissage.

Ce choix est adapté aux séries temporelles car il permet de modéliser des dépendances longues dans une fenêtre, sans les contraintes de récursivité des architectures de type RNN/LSTM.

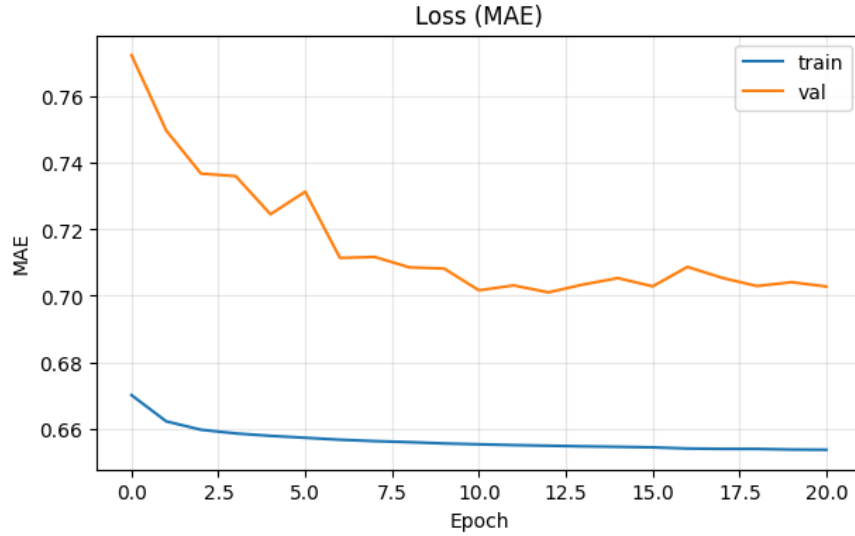


FIGURE 4.1 – Courbes d’apprentissage : évolution de la perte MAE sur l’ensemble d’entraînement (train) et de validation (val) au fil des époques.

4.4 Apprentissage : fonction de perte de reconstruction

4.4.1 Objectif d’optimisation

L’apprentissage consiste à minimiser une perte de reconstruction sur l’ensemble d’entraînement $\mathcal{X}$:

$$\min_{\theta, \phi} \frac{1}{|\mathcal{X}|} \sum_{X \in \mathcal{X}} \mathcal{L}(X, \hat{X}), \quad \text{où } \hat{X} = g_{\phi}(f_{\theta}(X)).$$

4.4.2 Perte MAE (Mean Absolute Error)

Dans ce projet, nous utilisons la MAE, définie pour une fenêtre $X \in \mathbb{R}^{T \times d}$ par :

$$\mathcal{L}_{\text{MAE}}(X, \hat{X}) = \frac{1}{Td} \sum_{t=1}^T \sum_{j=1}^d |X_{t,j} - \hat{X}_{t,j}|.$$

La MAE présente une robustesse intéressante vis-à-vis de pics ponctuels comparée à la MSE, et reste directement interprétable comme une erreur absolue moyenne.

4.4.3 Rôle de la compression latente

La contrainte $k \ll Td$ force l’encodeur à conserver l’information essentielle dans z :

$$z = f_{\theta}(X) \in \mathbb{R}^k.$$

Ainsi, l’espace latent devient une représentation compacte et adaptée à la visualisation après projection en 2D/3D.

4.5 Extraction du latent et visualisation par PCA

4.5.1 Extraction des représentations latentes

Après entraînement, pour chaque fenêtre X_t , on calcule :

$$z_t = f_\theta(X_t) \in \mathbb{R}^k.$$

En regroupant les latents, on obtient :

$$Z = \begin{bmatrix} z_0^\top \\ z_1^\top \\ \vdots \\ z_{N_w-1}^\top \end{bmatrix} \in \mathbb{R}^{N_w \times k},$$

où N_w est le nombre de fenêtres.

4.5.2 Projection PCA 2D pour la visualisation

Pour visualiser l'espace latent, nous appliquons une PCA sur Z et projetons en dimension 2 :

$$y_t = P(z_t - \bar{z}) \in \mathbb{R}^2,$$

où $P \in \mathbb{R}^{2 \times k}$ contient les deux directions principales et $\bar{z}$ est la moyenne des latents. Cette projection permet d'observer des structures (clusters, transitions) et d'interpréter l'évolution temporelle comme une trajectoire dans l'espace latent projeté.

4.6 Indicateur complémentaire : score MAE de reconstruction

4.6.1 Score par fenêtre

Pour un usage de diagnostic complémentaire, on définit un score de reconstruction par fenêtre :

$$s_t = \text{MAE}(X_t, \hat{X}_t) = \frac{1}{Td} \sum_{u=1}^T \sum_{j=1}^d |X_{t,u,j} - \hat{X}_{t,u,j}|.$$

Un score élevé indique que la fenêtre est difficile à reconstruire, suggérant un comportement éloigné du régime principal appris par l'autoencodeur.

4.6.2 Seuil d'entraînement et seuil robuste (MAD)

Le seuil issu de l'entraînement est noté τ_{train} (estimé sur les scores du train). Pour améliorer la robustesse en test, on peut définir un seuil adaptatif basé sur la médiane et la

MAD, calculé sur une baseline :

$$\text{MAD}(s) = \text{median}(|s - \text{median}(s)|), \quad \tau_{\text{MAD}} = \text{median}(s_{\text{base}}) + k \cdot \text{MAD}(s_{\text{base}}).$$

Le seuil final utilisé est :

$$\tau = \max(\tau_{\text{train}}, \tau_{\text{MAD}}).$$

4.7 Décision : NOMINAL / DÉGRADATION / PANNE

Soit $s_{\text{max}} = \max_t s_t$ le score maximal observé sur le fichier. Une règle simple à trois niveaux est définie par :

$$\text{Statut} = \begin{cases} \text{PANNE} & \text{si } s_{\text{max}} > \tau, \\ \text{DÉGRADATION} & \text{si } 0.8\tau < s_{\text{max}} \leq \tau, \\ \text{NOMINAL} & \text{si } s_{\text{max}} \leq 0.8\tau. \end{cases}$$

En complément, on calcule le ratio de fenêtres dépassant le seuil :

$$r = \frac{1}{N_w} \sum_{t=1}^{N_w} \mathbb{K}[s_t > \tau],$$

ce qui permet de caractériser si l'écart est ponctuel (pic isolé) ou persistant (dérive progressive).

4.8 Synthèse et lien avec le sujet PFE

La méthodologie proposée répond au sujet « *Autoencoders profonds pour la réduction de dimension et la visualisation de données complexes* » :

- **Réduction de dimension** : l'encodeur apprend $X_t \mapsto z_t$ avec $k \ll Td$;
- **Visualisation** : les latents z_t sont projetés en 2D via PCA pour analyser la structure ;
- **Analyse complémentaire** : la MAE et des seuils robustes aident à repérer les écarts au comportement appris.

Chapitre 5

Résultats et analyse

5.1 Objectif des expériences et protocole

Ce chapitre présente les résultats obtenus avec le Transformer Autoencoder entraîné sur les fenêtres MAV normalisées (Chapitre 3) selon la méthodologie définie au Chapitre 4. L'objectif principal est d'évaluer :

- la **qualité de la représentation latente** pour la **réduction de dimension** et la **visualisation** ;
- la cohérence entre **latent** et **erreur de reconstruction** (MAE) ;
- la robustesse de l'approche sur plusieurs fichiers (début vs fin de vie du roulement).

Les tests sont conduits sur des fichiers du jeu IMS Bearing, en comparant typiquement des segments *précoces* (régime nominal) et des segments *tardifs* (régime dégradé voire proche de la panne). Pour chaque fichier, on calcule :

$$s_t = \text{MAE}(X_t, \hat{X}_t), \quad s_{\max} = \max_t s_t, \quad r = \frac{1}{N_w} \sum_{t=1}^{N_w} \mathbb{I}[s_t > \tau].$$

Le seuil τ est défini comme :

$$\tau = \max(\tau_{\text{train}}, \tau_{\text{MAD}}),$$

et le statut final est déterminé à partir de $s_{\max}$ et du ratio r (Chapitre 4).

5.2 Qualité de représentation : comportement du latent (PCA 2D)

5.2.1 Projection PCA 2D et séparation des régimes

Une première manière d'évaluer l'espace latent est de projeter les représentations $z_t \in \mathbb{R}^k$ en dimension 2 via PCA :

$$y_t = P(z_t - \bar{z}) \in \mathbb{R}^2.$$

La Figure 5.1 montre la projection PCA 2D des latents, avec une coloration des points en fonction d'un critère d'écart (par exemple l'étiquette *normal/anomalie* dérivée de $s_t > \tau$).

On observe typiquement :

- un regroupement dense correspondant au régime nominal ;
- une zone plus dispersée et/ou décalée correspondant à des fenêtres atypiques ;
- des transitions progressives, cohérentes avec une dérive du comportement.

Variance expliquée PCA: [0.74422723 0.13090155]

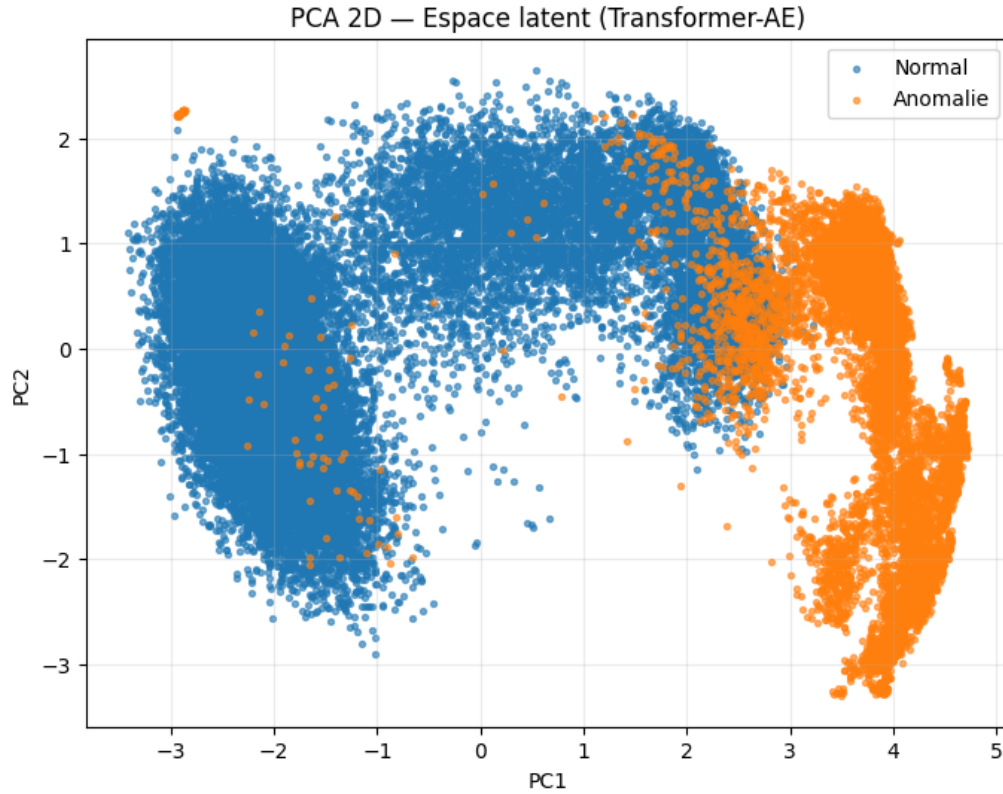


FIGURE 5.1 – Projection PCA 2D des représentations latentes z_t apprises par le Transformer Autoencodeur. Les points sont colorés selon le statut (normal vs anomalie) défini à partir du score MAE et du seuil τ . Cette visualisation illustre la capacité du latent à organiser les fenêtres en régimes cohérents et à faire émerger des comportements atypiques.

5.2.2 Analyse : interprétation du latent

La structure observée dans le plan PCA suggère que la compression non linéaire apprise par l'autoencodeur capture des variations significatives du signal vibratoire. Mathématiquement, si deux fenêtres X_a et X_b correspondent à des régimes proches, on s'attend à avoir :

$$\|z_a - z_b\|_2 \text{ faible.}$$

À l'inverse, lorsque le comportement change, les latents se déplacent dans l'espace et la projection PCA reflète un déplacement global (ou l'apparition de nouveaux groupes). Cette propriété soutient l'objectif du PFE : **obtenir une représentation compacte et visualisable** de données complexes.

5.3 Étude sur plusieurs fichiers : début vs fin (dérive temporelle)

5.3.1 Comparaison qualitative

En comparant un fichier situé en début de séquence (fonctionnement nominal) et un fichier tardif (fonctionnement dégradé), on observe généralement :

- une augmentation de la variabilité des descripteurs MAV ;
- un déplacement ou une dispersion accrue dans le plan PCA du latent ;
- une augmentation du score MAE, traduisant une moins bonne reconstruction.

5.3.2 Métriques synthétiques (à reporter selon tes tests)

Pour chaque fichier testé, il est pertinent de reporter les métriques suivantes :

$$s_{\max}, \quad \bar{s} = \frac{1}{N_w} \sum_{t=1}^{N_w} s_t, \quad r = \frac{1}{N_w} \sum_{t=1}^{N_w} \mathbb{I}[s_t > \tau], \quad \tau.$$

Remarque : dans ce mémoire, ces valeurs peuvent être reprises directement depuis les sorties du notebook et/ou les indicateurs du dashboard.

5.4 Analyse du score MAE et du ratio

5.4.1 Évolution du score et franchissement de seuil

La Figure 5.2 illustre l'évolution du score MAE au fil des fenêtres, ainsi que le seuil τ . On observe que les phases de dérive se traduisent par une augmentation progressive de s_t . Le franchissement de τ met en évidence des fenêtres que le modèle reconstruit difficilement :

$$s_t > \tau \Rightarrow X_t \text{ éloigné du comportement appris.}$$

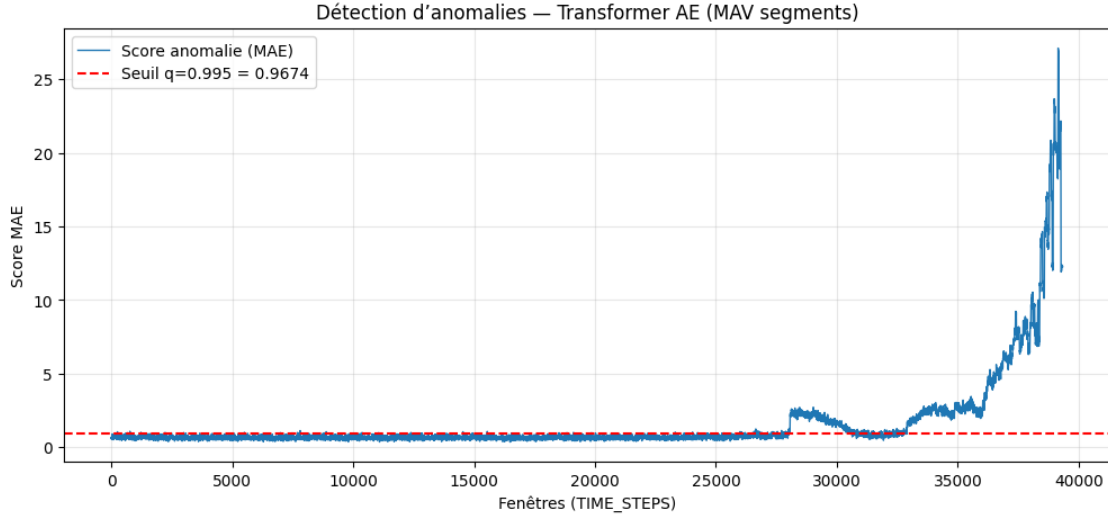


FIGURE 5.2 – Score d’erreur de reconstruction (MAE) en fonction des fenêtres (TIME_STEPS) et seuil τ . Les dépassements du seuil indiquent des segments dont la reconstruction s’éloigne du régime nominal appris.

5.4.2 Rôle du ratio r

Le ratio

$$r = \frac{1}{N_w} \sum_{t=1}^{N_w} \mathbb{I}[s_t > \tau]$$

aide à distinguer :

- des pics isolés (faible r) pouvant provenir de bruit ou d’événements ponctuels ;
- une dérive persistante (fort r) traduisant une évolution structurelle du signal.

Ainsi, r complète l’analyse basée sur $s_{\max}$ et améliore l’interprétabilité de la décision.

5.5 Cas limites et phénomènes observés

5.5.1 Seuil adaptatif trop élevé

Un cas défavorable survient si le seuil adaptatif τ_{MAD} est calculé sur une portion déjà dégradée : la médiane et la MAD augmentent, et le seuil devient trop permissif. Pour limiter cela, le calcul est effectué sur une *baseline* (début du fichier), ce qui revient à estimer la statistique sur une zone supposée proche du nominal.

5.5.2 Dataset shift et sensibilité aux conditions

Les mesures vibratoires peuvent varier selon :

- les conditions de charge et de vitesse,
- le capteur,
- la distribution statistique des signaux (bruit, environnement).

Ce *dataset shift* peut affecter le score MAE et la structure du latent. Une solution consiste à recalibrer le scaler et/ou les seuils sur un régime nominal récent, ou à utiliser un latent plus régularisé.

5.6 Discussion : cohérence latent + reconstruction

Un point important est la cohérence entre :

- le **latent** : qui fournit une carte visuelle des régimes et transitions ;
- la **MAE** : qui quantifie l'écart de reconstruction.

Dans les phases de dérive, on observe généralement un déplacement dans la projection PCA et une augmentation du score MAE. Cette cohérence renforce la validité de l'approche : le latent explique *où* se situent les fenêtres dans l'espace appris, tandis que la MAE indique *à quel point* elles s'en éloignent. Cela répond directement à l'objectif du PFE : **réduire la dimension et visualiser des données complexes**, tout en conservant un indicateur complémentaire pour interpréter les comportements atypiques.

Chapitre 6

Application et déploiement (outil de visualisation)

6.1 Objectifs de l'outil de visualisation

L'objectif de ce chapitre est de présenter l'outil logiciel développé pour exploiter concrètement la méthode proposée dans ce PFE : **apprendre un espace latent de faible dimension** via un autoencodeur profond, puis **visualiser** les données complexes dans cet espace (projection PCA), tout en fournissant un **indicateur interprétable** basé sur l'erreur de reconstruction (MAE).

L'application Streamlit répond à trois besoins principaux :

- **Exploration** : charger un fichier de signaux vibratoires multi-capteurs (IMS Bearing) et visualiser un aperçu.
- **Visualisation par réduction de dimension** : projeter les représentations latentes en 2D via PCA afin d'observer des régimes, transitions et comportements atypiques.
- **Aide au diagnostic** : calculer un score d'anomalie s (MAE) et une décision *NOMINAL* / *DÉGRADATION* / *PANNE* à partir de seuils robustes.

6.2 Architecture logicielle et pipeline de traitement

6.2.1 Choix technologiques

Le développement s'appuie sur :

- **Streamlit** pour l'interface web (upload, paramètres, affichage).
- **TensorFlow/Keras** pour le chargement et l'inférence du Transformer Autoencoder.
- **scikit-learn** pour la normalisation (scaler) et la PCA.
- **Plotly** pour des visualisations interactives (courbes MAE, projection 2D).

6.2.2 Entrées, assets et dépendances

L'application nécessite trois artefacts produits lors de l'entraînement :

- un modèle sauvegardé : `transformer_bearing_anomaly_detection.keras`;
- un normaliseur : `scaler_bearing.gz`;

- un fichier de configuration : `threshold_transformer.json` (seuil τ , `TIME_STEPS`, etc.).

6.2.3 Pipeline complet côté application

Le pipeline implémenté dans le dashboard suit une chaîne cohérente avec la méthodologie des Chapitres 3 et 4 :

1. **Lecture** du fichier (format TAB, 4 colonnes).
2. **Segmentation** en blocs de taille L puis extraction MAV :

$$\text{MAV}(m) = \frac{1}{L} \sum_{n=1}^L |s[n]|.$$

3. **Fenêtrage temporel** : construction des séquences $X_t \in \mathbb{R}^{T \times d}$.
4. **Normalisation** : $X_t \leftarrow \text{Scaler}(X_t)$.
5. **Inférence** : reconstruction $\hat{X}_t$ via le Transformer Autoencoder.
6. **Score MAE** par fenêtre :

$$s(X_t) = \frac{1}{Td} \sum_{i=1}^T \sum_{j=1}^d |X_{t,i,j} - \hat{X}_{t,i,j}|.$$

7. **Décision** par seuils (train + adaptatif MAD) et classification en 3 niveaux.
8. **Visualisation latent** : extraction z_t puis PCA en 2D.

6.3 Interface utilisateur : organisation et paramètres

6.3.1 Organisation générale

L'interface est structurée en deux zones :

- **Sidebar** : état des assets (modèle/scaler/seuil), upload fichier, paramètres (segment, seuil décision, MAD, debug).
- **Zone principale** : aperçu des données, bouton de lancement, métriques, courbes MAE, visualisation PCA du latent.

6.3.2 Paramètres contrôlables

Les paramètres exposés permettent d'ajuster la sensibilité et l'adaptation aux fichiers :

- taille de segment L (extraction MAV) ;
- seuil de décision en ratio : $\rho = \frac{1}{N} \sum_{t=1}^N \mathbb{1}[s(X_t) > \tau]$;
- activation du seuil adaptatif MAD : $\tau_{\text{MAD}} = \text{med}(s) + k \cdot \text{MAD}(s)$;

- activation du mode debug (affichage des statistiques et tailles intermédiaires).

6.3.3 Sorties et interprétation

L'application affiche :

- $s_{\max} = \max_t s(X_t)$, indicateur de sévérité ;
- le seuil utilisé τ (train ou adaptatif) ;
- le ratio ρ de fenêtres au-dessus du seuil ;
- un **statut** :

NOMINAL / DÉGRADATION / PANNE.

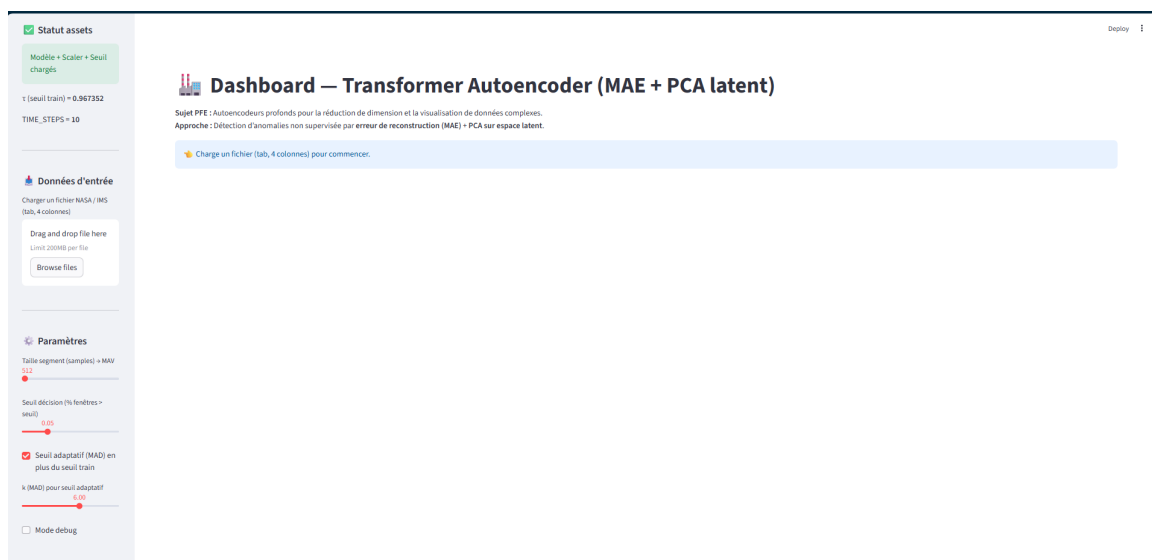


FIGURE 6.1 — Interface Streamlit : chargement des assets et zone d'upload.



FIGURE 6.2 — Exemple : statut NOMINAL avec score MAE faible et ratio sous le seuil.

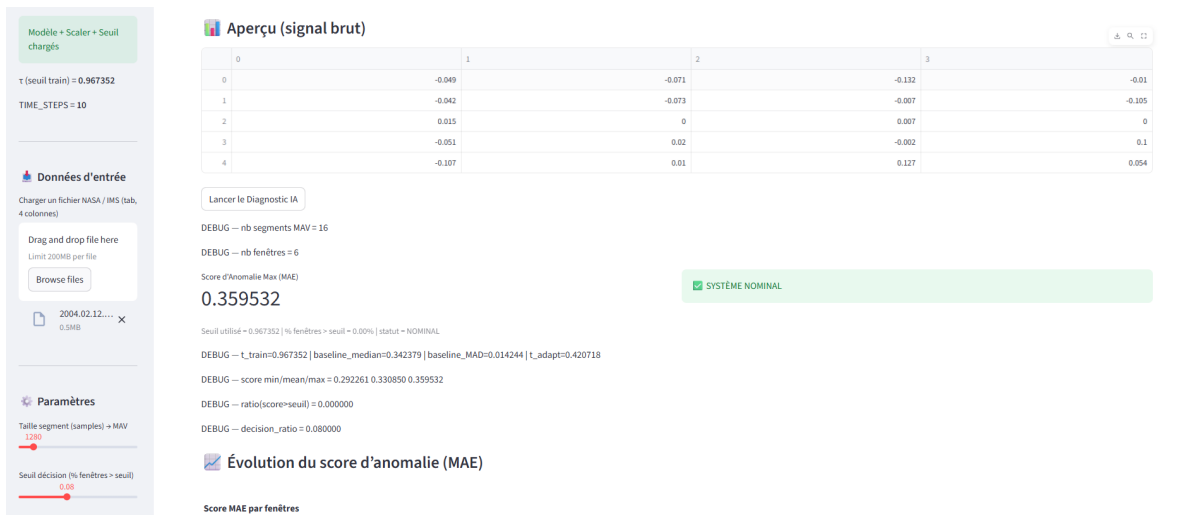


FIGURE 6.3 – Capture du dashboard Streamlit en fonctionnement nominal : chargement des assets (modèle, scaler, seuil), aperçu du signal, score MAE maximal, et informations du mode debug (segments MAV, fenêtres, seuils).

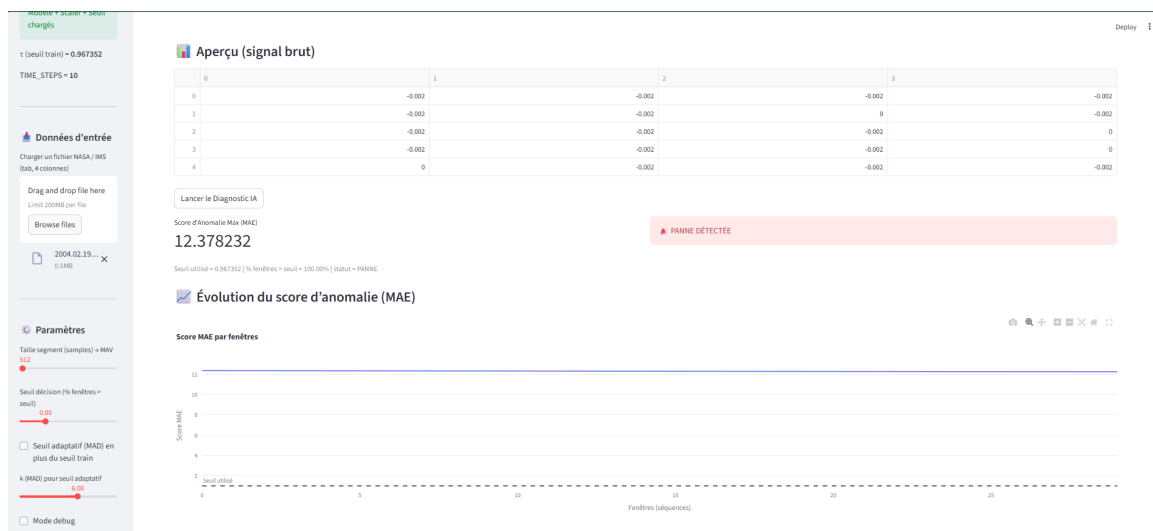


FIGURE 6.4 – Exemple : statut PANNE avec score MAE élevé et forte proportion de fenêtres au-dessus du seuil.



FIGURE 6.5 – Projection PCA 2D des représentations latentes : séparation visuelle entre régimes normaux et atypiques.

6.4 Robustesse : chargement du modèle et gestion des erreurs

6.4.1 Chargement du Transformer Autoencoder avec couche personnalisée

Le modèle utilise un bloc Transformer défini comme couche personnalisée. Pour garantir le chargement (*désérialisation*) dans l'application, la couche est enregistrée et passée à `custom_objects` lors de `load_model`. Le chargement se fait avec `compile=False` afin d'éviter les dépendances à l'optimizer et aux objets de compilation.

6.4.2 Validation des entrées

Avant tout calcul, l'application vérifie :

- présence des fichiers requis (modèle, scaler, seuil) ;
- format du fichier upload (TAB) ;
- nombre de colonnes attendu : $d = 4$ (4 capteurs) ;
- longueur suffisante pour produire des segments puis des fenêtres ($m > T$).

6.4.3 Extraction du latent : cas où la couche n'existe pas

Pour effectuer la PCA, l'application tente d'extraire z_t depuis une couche nommée `latent`. Si la couche n'est pas présente, une stratégie de repli consiste à :

- soit désactiver la PCA latent et informer l'utilisateur ;

- soit projeter une représentation alternative (ex. entrée aplatie) uniquement à titre illustratif.

Dans ce PFE, la recommandation est de conserver explicitement une couche **latent** lors de la sauvegarde du modèle pour garantir la cohérence *réduction de dimension* $\rightarrow$ *visualisation*.

6.5 Guide d'utilisation (pas à pas)

1. Lancer l'application : `streamlit run app.py`
2. Vérifier le chargement des assets (message vert dans la sidebar).
3. Importer un fichier IMS Bearing (tabulation, 4 colonnes).
4. Ajuster L (segments MAV) et T (TIME_STEPS issu du training).
5. Cliquer sur **Lancer le Diagnostic IA**.
6. Interpréter :
 - la courbe $s(X_t)$ et la position du seuil ;
 - la PCA du latent (structure / séparation / trajectoires) ;
 - le statut final et les métriques ($s_{\max}$, ρ).

6.6 Déploiement et reproductibilité

6.6.1 Exécution locale

Le mode standard est une exécution locale, adaptée aux tests et démonstrations.

6.6.2 Reproductibilité

Pour assurer la reproductibilité, il est recommandé de fournir :

- un fichier `requirements.txt` (Streamlit, TensorFlow, scikit-learn, joblib, plotly, pandas, numpy) ;
- les versions principales (TensorFlow/Keras) utilisées lors de la sauvegarde du modèle ;
- un exemple de fichier d'entrée et les paramètres par défaut (segment, seuil décision, k MAD).

6.7 Limites et perspectives (côté application)

- **Latent 2D via PCA** : utile et simple, mais peut être enrichi par UMAP/t-SNE (à étudier).

- **Seuil adaptatif** : robuste mais peut devenir trop permissif si la baseline est déjà dégradée ; une baseline strictement nominale est préférable.
- **Scalabilité** : sur très gros fichiers, prévoir un échantillonnage, un streaming par blocs, ou une réduction de résolution temporelle.

Chapitre 7

Conclusion et perspectives

7.1 Bilan : réduction de dimension et visualisation de données complexes

L'objectif central de ce travail était de proposer une approche basée sur des **autoencodeurs profonds** afin d'obtenir une **représentation compacte** (réduction de dimension) et **visualisable** de données complexes et de grande dimension. Dans le cadre de séries temporelles vibratoires multi-capteurs (jeu IMS Bearing), la difficulté principale provient (i) de la nature temporelle des signaux, (ii) de la variabilité des régimes de fonctionnement et (iii) de l'absence d'annotations exhaustives.

L'approche retenue repose sur l'apprentissage d'une application non linéaire

$$f_{\theta} : \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^k, \quad k \ll Td,$$

qui projette chaque fenêtre temporelle $X_t \in \mathbb{R}^{T \times d}$ vers une variable latente $z_t = f_{\theta}(X_t)$, puis reconstruit une approximation $\hat{X}_t = g_{\phi}(z_t)$. La réduction de dimension est donc réalisée par le goulot d'étranglement latent, tandis que la **visualisation** est obtenue en projetant $\{z_t\}$ vers $\mathbb{R}^2$ (par PCA) afin d'observer la structure globale, les regroupements et les transitions.

Les résultats montrent que l'espace latent appris fournit une lecture interprétable de l'évolution des données : les fenêtres associées à un comportement nominal se concentrent dans des régions spécifiques de l'espace projeté, tandis que des comportements atypiques apparaissent sous forme de trajectoires qui s'éloignent progressivement de ces régions. Cette observation renforce l'intérêt des autoencodeurs comme outil de **réduction de dimension orientée données** et de **visualisation** pour l'exploration de signaux industriels complexes.

7.2 Apports du Transformer Autoencoder

Un apport important de ce projet réside dans l'utilisation d'un **Transformer Autoencoder** pour encoder des fenêtres temporelles multi-capteurs. Grâce au mécanisme d'attention, le modèle peut capter des dépendances internes dans la fenêtre et apprendre une représentation latente adaptée à la structure des signaux.

Sur le plan formel, le modèle apprend une représentation z_t minimisant une perte de

reconstruction. Dans ce travail, la perte de référence est de type MAE :

$$\mathcal{L}(\theta, \phi) = \frac{1}{N} \sum_{t=1}^N \text{MAE}(X_t, \hat{X}_t) = \frac{1}{N} \sum_{t=1}^N \frac{1}{Td} \sum_{i=1}^T \sum_{j=1}^d |X_t(i, j) - \hat{X}_t(i, j)|.$$

Cette optimisation favorise un espace latent qui capture les facteurs dominants expliquant les observations, tout en fournissant un indicateur complémentaire de cohérence via l'erreur de reconstruction.

Enfin, l'intégration dans un **dashboard Streamlit** constitue un apport applicatif : elle rend l'approche exploitable pour l'analyse et la visualisation, en permettant de charger un fichier, paramétrer la segmentation, afficher la courbe MAE et visualiser l'espace latent en 2D.

7.3 Perspectives

Plusieurs extensions peuvent améliorer la qualité de la réduction de dimension, la visualisation et l'exploitation du latent :

7.3.1 Visualisations non linéaires : UMAP et t-SNE sur l'espace latent

La PCA fournit une projection linéaire. Une première perspective naturelle est d'utiliser une méthode non linéaire, en particulier **UMAP**, appliquée aux vecteurs latents $\{z_t\}$. UMAP peut préserver davantage la structure locale et améliorer la séparation visuelle de régimes proches. Une comparaison systématique PCA vs UMAP (sur les mêmes latents) permettrait d'évaluer la stabilité des regroupements et la lisibilité des transitions.

7.3.2 Latents 3D et exploration interactive

La visualisation en 2D peut perdre de l'information lorsque k est important. Une extension consiste à projeter en **3D** (PCA 3D ou UMAP 3D), et à fournir une exploration interactive (rotation, survol de points, encodage temporel par couleur). Cela aiderait à analyser la trajectoire temporelle dans l'espace latent et les zones de transition.

7.3.3 Clustering et segmentation automatique dans le latent

L'espace latent peut servir de base à une segmentation automatique des régimes via des méthodes non supervisées :

k-means sur $\{z_t\}$, ou clustering hiérarchique, ou DBSCAN.

L'objectif serait d'identifier des clusters interprétables (nominal, transition, anomalie), et de suivre l'apparition de nouveaux clusters comme signe de dérive. On peut également étudier

des mesures de séparation inter-clusters et de compacité intra-cluster pour quantifier la qualité de la représentation.

7.3.4 Suivi temporel et détection de dérive (drift)

Au-delà d'un score instantané, une perspective importante est d'intégrer un **suivi temporel** : moyennes glissantes du MAE, statistiques robustes sur des fenêtres, ou distances progressives dans le latent :

$$\Delta_t = \|z_t - z_{t-1}\|_2, \quad D_t = \|z_t - \mu_{\text{nominal}}\|_2.$$

Ces indicateurs permettent de caractériser une dérive lente (dégradation) avant qu'une panne ne soit manifeste.

7.3.5 Améliorations méthodologiques

Enfin, plusieurs améliorations du modèle peuvent être envisagées :

- apprentissage contrastif ou régularisation du latent pour stabiliser les trajectoires ;
- choix plus systématique de k (dimension latente) via compromis reconstruction / séparabilité ;
- validation croisée sur plusieurs roulements / plusieurs runs pour réduire le risque de *dataset shift*.

7.4 Conclusion générale

Ce projet a montré qu'un Transformer Autoencoder permet d'apprendre un espace latent compact, utile pour la **réduction de dimension** et la **visualisation** de signaux industriels complexes. La projection des latents fournit une interprétation qualitative de l'évolution des régimes, et l'erreur de reconstruction apporte un complément quantitatif exploitable pour l'analyse. Les perspectives proposées ouvrent la voie vers des visualisations plus expressives (UMAP/3D), une exploitation plus automatique (clustering) et un suivi plus robuste dans le temps (détection de dérive), en cohérence directe avec les objectifs du sujet PFE.